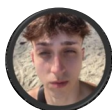


FIAP — ATIVIDADE MOBILE HYBRID APP E A SOCIEDADE 5.0

Smart HAS / Digital 360

Fase 5 — Back-end Spring Boot + Dashboard Angular



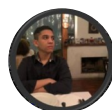
Eduardo Andrade Martins Vasques — RM 556970



Otavio Ramos dos Santos Souza — RM 550361



Enzo Miranda Ward de Paiva — RM 557632



Rafael Pinto de Albuquerque — RM 559136



Guilherme Leoni Vidigal Tiburcio — RM 557500

Repositório (GitHub): https://github.com/DevRafonalde/digital360_flutter

Vídeo de apresentação (YouTube, não listado): [LINK DO VÍDEO – preencher após publicar]

Sumário

1. Escolha da stack tecnológica e justificativa
2. Back-end escalável com Spring Boot
3. Integração web com Angular
4. Limitações conhecidas e próximos passos
5. Como executar o projeto completo

1. Escolha da stack tecnológica e justificativa

1.1 Decisão: aprimoramento da solução Flutter existente (Opção B)

O grupo optou por **não migrar para React Native** e por **aprimorar significativamente** a solução Flutter já construída nas fases anteriores. A decisão não foi por comodismo: foi uma escolha técnica deliberada, baseada no estado real do projeto.

O app Digital 360 hoje já é uma base madura: mais de 30 telas, arquitetura em camadas (UI → Provider → Service → Model, equivalente a um MVVM), gerenciamento de estado com Provider (um `ChangeNotifier` por domínio), integração com dois web services externos reais (Open-Meteo e ViaCEP), Firebase/FCM para notificações push, mapas (Google Maps no mobile e OpenStreetMap na web), autenticação JWT real com renovação automática de sessão, login biométrico, acessibilidade dedicada ao público-alvo (idosos e pessoas com baixo letramento digital), conformidade com LGPD e mais de 80 testes automatizados rodando em pipeline de CI. Reescrever essa base do zero em React Native jogaria fora um investimento de engenharia considerável sem nenhum ganho técnico proporcional: o desafio desta fase não é "qual framework de UI usar", e sim "como evoluir a solução com integrações reais e arquitetura escalável", que é exatamente o que a Opção B permite fazer com o tempo disponível.

1.2 Alinhamento com os objetivos do projeto

O projeto Smart HAS, na frente Sociedade 5.0, existe para reduzir a exclusão digital de um público que depende de uma experiência confiável e simples. Trocar de stack mobile no meio do caminho aumentaria o risco de regressão justamente nas telas pensadas para esse público (leitura em voz alta, busca por voz, tema de alto contraste, onboarding guiado). Manter o Flutter e investir a energia desta fase em **conectividade real** (um backend Java robusto) e em **visibilidade administrativa** (um dashboard Angular) atende de forma mais direta o objetivo da fase: evoluir o sistema "de forma inteligente e conectada".

1.3 Melhoria implementada no Flutter nesta atividade

"Aprimorar" não ficou só na justificativa: o app ganhou uma funcionalidade nova de verdade, com um refactor de arquitetura por trás dela.

Nova funcionalidade: em Configurações → "Conexão (avançado)", o usuário agora escolhe, em tempo real e sem recompilar o app, qual backend o app conversa: **Mock** (demonstração offline), **Python** (backend real do time, cobre todo o app) ou **Java** (o backend Spring Boot construído nesta fase, cobrindo o contrato completo). A escolha fica salva entre sessões.

Refatoração de arquitetura: até então, `ApiConstants.useMock` e `ApiConstants.baseUrl` eram `static const` — fixos em tempo de compilação, só trocáveis editando o código-fonte e recompilando. Isso não fazia mais sentido a partir do momento em que o projeto passou a ter **dois** backends reais (Python e Java), além do mock. Os dois campos viraram `static` (não-const), com um novo método `ApiConstants.aplicarAmbiente()` que ajusta os dois de uma vez a partir de um enum `AmbienteBackend`. O ganho de projeto: nenhum dos 65 pontos de `api_service.dart` que já liam `ApiConstants.useMock/baseUrl` precisou mudar — a sintaxe de leitura de um campo estático é a mesma seja ele `const` ou não. É uma inversão de dependência de baixo custo: a decisão de qual ambiente usar saiu do código fixo e virou uma preferência

do usuário, persistida com `SharedPreferences`, seguindo o mesmo padrão já usado pelo `SettingsProvider` para tema e tamanho de fonte.

Verificado de verdade: `dart analyze lib test` → nenhum problema encontrado. `flutter test` → as 85 verificações da suíte (as 80 anteriores + 5 novas cobrindo o seletor de ambiente e a persistência da escolha) passam, sem nenhuma regressão.

1.4 Roadmap tecnológico

Já concluído (fases anteriores):

- Fase 1-3: prototipação, arquitetura e camada de conectividade/NOC (Kotlin, depois reimplementado em Flutter).
- Fase 4: app completo em Flutter (Provider, Google Maps/OpenStreetMap, Firebase/FCM, 2 web services reais).
- Pós-Fase 4 (evolução contínua do grupo, antes desta atividade): redesign visual, backend real em Python/FastAPI com persistência (SQLite) e autenticação JWT real, cursos comunitários, fórum, gamificação, modo cuidador, indicação de amigos, marketplace de tutores, LGPD, login biométrico, acessibilidade adicional e pipeline de CI.

Sendo feito nesta atividade (Fase 5):

- Back-end **completo** em **Java + Spring Boot** (`backend-java/`), cobrindo o contrato inteiro que o app consome: autenticação (LGPD, indicação), cursos oficiais e comunitários, guia de serviços, AI Logistics Extension, fórum, gamificação, modo cuidador, marketplace de tutores, recomendações e tendências — com persistência própria, autenticação/autorização JWT por perfil e por dono do recurso, e documentação OpenAPI/Swagger.
- Dashboard administrativo em **Angular** (`dashboard/`), consumindo essa mesma API REST.
- No app Flutter: novo seletor de ambiente de backend (Mock/Python/Java) em Configurações, com o refactor de `ApiConstants` que o sustenta.

Planejado para as próximas fases:

- Decidir, junto com o time, se os dois backends (Python e Java) convergem para um só ou se seguem servindo públicos diferentes.
- Evoluir os motores de risco e recomendação de heurística por regras para modelos de Machine Learning treinados com dados reais de uso.
- Expandir o dashboard Angular para cobrir também comunidade, gamificação e painel de impacto, se isso agregar valor.
- Suíte de testes JUnit (backend) e Karma/Jasmine (dashboard), como hardening final.

2. Back-end escalável com Spring Boot

2.1 Necessidade real atendida

A API cobre o domínio completo de negócio do Smart HAS consumido pelo app: autenticação (com LGPD e indicação de amigos), trilhas de curso oficiais e comunitárias, guia de serviços públicos, a AI Logistics Extension (pedidos, risco, reagendamento, feedback, assistente) e as áreas de comunidade — fórum, gamificação, modo cuidador, marketplace de tutores, recomendações e tendências. O contrato de cada endpoint foi conferido contra o código atual do app (`lib/data/services/api_service.dart`) e contra o backend Python de referência (`backend/app/main.py`, `db_models.py`, `heuristics.py`), garantindo que nomes de campos, rotas e regras de negócio (por exemplo, o campo de senha chama-se `senhaUser`, não `senha`; um curso comunitário exige `isTutor=true`; recomendações e gamificação só são visíveis para o próprio usuário) batem com o que o app realmente espera.

2.2 Arquitetura

Camadas separadas por responsabilidade, seguindo o padrão Controller → Service → Repository → Model:

```
backend-java/src/main/java/com/smarthas/backend/
  config/      -> SecurityConfig, OpenApiConfig, DataSeeder
  security/    -> JwtService, JwtAuthenticationFilter, CustomUserDetailsService
  model/       -> Usuario, Curso, Servico, PedidoLogistico, Evento, PerguntaForum,
                RespostaForum, VinculoCuidador, AvaliacaoCurso, FeedbackEntrega
  repository/  -> 10 interfaces Spring Data JPA
  dto/         -> Request/Response separados das entidades
  service/     -> regras de negócio + motores heurísticos (RiscoEngine,
                RecomendacaoEngine, TendenciaEngine, RascunhoCursoEngine,
                GamificacaoEngine)
  controller/  -> 13 controllers REST (um por área de domínio)
  exception/   -> ApiException + GlobalExceptionHandler
```

2.3 Autenticação e autorização

- **Autenticação:** JWT stateless (access + refresh token), assinado com HMAC, senha com hash BCrypt.
- **Autorização por perfil:** ADMIN (escreve) x USUARIO (lê) — leitura liberada para qualquer usuário autenticado, escrita em serviços e pedidos restrita a ADMIN.
- **Autorização por atributo/dono do recurso:** publicar curso comunitário exige `isTutor=true`; recomendações e gamificação só o próprio usuário vê; o resumo de um idoso só é visível para quem tem vínculo de cuidador com ele.
- **Renovação de sessão:** POST `/auth/refresh` emite novo par de tokens sem exigir novo login.

2.4 Persistência e integração

- Banco H2 em arquivo, via Spring Data JPA, sem servidor externo.
- `DataSeeder` popula dados de demonstração idênticos ao mock do app.

- Validação com Bean Validation e tratamento de erros centralizado (GlobalExceptionHandler).
- CORS restrito por lista explícita de origens (não usa "*").

2.5 Endpoints (33 ao todo — contrato completo do app)

Autenticação e perfil

Método	Rota	Auth	Descrição
POST	/auth/usuarios/login	público	Login (nomeUser + senhaUser)
POST	/auth/usuarios/registrar	público	Cadastro (exige aceite da política de privacidade)
POST	/auth/refresh	público	Renovação de sessão
POST	/perfil/tornar-tutor	Bearer	Auto-atribuição de tutor
GET	/indicacoes/minhas	Bearer	Código de indicação e total de indicados
DELETE	/perfil/conta	Bearer	Exclusão da própria conta (LGPD)

Cursos, serviços e AI Logistics

Método	Rota	Auth	Descrição
GET	/cursos	Bearer	Lista trilhas publicadas
GET	/cursos/meus	Bearer	Cursos do próprio tutor
POST	/cursos	Bearer	ADMIN cadastra oficial, ou tutor publica comunitário
POST	/cursos/gerar-rascunho	Bearer (tutor)	Rascunho de módulos por template
PUT/DELETE	/cursos/{id}	Bearer ADMIN	Editar/remover do catálogo
GET	/servicos	Bearer	Lista guia de serviços públicos

Método	Rota	Auth	Descrição
POST/PUT/DELETE	/servicos[/{id}]	Bearer ADMIN	CRUD do guia
GET	/pedidos	Bearer	Lista pedidos logísticos monitorados
POST/PUT/DELETE	/pedidos[/{id}]	Bearer ADMIN	CRUD de pedidos
POST	/entregas/{id}/recalcular-risco	Bearer	Recalcula o risco (aceita impactoClima)
POST	/pedidos/{id}/reagendar	Bearer	Reagenda uma entrega atrasada
POST	/feedback-entrega	Bearer	Registra feedback pós-entrega
POST	/assistente-logistico/pergunta	Bearer	Assistente conversacional por regras

Comunidade, gamificação e inteligência

Método	Rota	Auth	Descrição
POST	/eventos	Bearer	Registra uso (alimenta recomendação/tendências/gamificação)
GET	/recomendacoes/{userId}	Bearer (dono)	Top 3 recomendações heurísticas
GET	/tendencias	Bearer	Picos de uso por categoria
POST/GET	/forum/perguntas	Bearer	Criar/listar perguntas
GET	/forum/perguntas/{id}	Bearer	Detalhe + respostas
POST	/forum/perguntas/{id}/respostas	Bearer	Responder
GET	/gamificacao/{userId}/resumo	Bearer (dono)	Cursos concluídos, sequência, pontos
GET	/gamificacao/ranking	Bearer	Top 10

Método	Rota	Auth	Descrição
GET	/metricas/impacto	Bearer	Números agregados da base atual
POST	/cuidador/vincular	Bearer	Vínculo somente-leitura com um idoso
GET	/cuidador/vinculos	Bearer	Vínculos do cuidador
GET	/cuidador/{idosoId}/resumo	Bearer (com vínculo)	Progresso do idoso
POST	/cursos/{id}/avaliar	Bearer	Avalia um curso
GET	/cursos/{id}/avaliacoes	Bearer	Avaliações de um curso
GET	/tutores/{autorId}	Bearer	Perfil público do tutor

2.6 Documentação

A API é documentada automaticamente via **springdoc-openapi**, disponível em <http://localhost:8080/swagger-ui.html> assim que o backend sobe.

Validado rodando de verdade (não só por leitura de código): `mvn compile` e `mvn spring-boot:run` executados com JDK 21 + Maven 3.9.14. Login, CRUD, recálculo de risco (incluindo o parâmetro opcional `impactoClima`), reagendamento e a autorização por perfil (ADMIN x USUARIO) foram exercitados via `curl` contra o servidor real, com sucesso. Além do núcleo, os fluxos de comunidade também foram testados ao vivo: tutor publicando curso, eventos alimentando gamificação/recomendações, fórum, cuidador, avaliação de curso e exclusão de conta (LGPD).

3. Integração web com Angular

3.1 Objetivo

Um painel administrativo que consome **a mesma API REST** usada pelo app mobile (Parte 2), permitindo a um gestor visualizar e manter cursos, serviços e pedidos sem precisar abrir o aplicativo mobile.

3.2 Arquitetura

Angular standalone components (sem NgModule), organizados por responsabilidade:

```
dashboard/src/app/  
  core/  
    models/      -> Curso, Servico, PedidoLogistico, Usuario (TypeScript)  
    services/    -> AuthService, CursoService, ServicoService, PedidoService  
    interceptors/ -> authInterceptor (injeta o Bearer token)  
    guards/      -> authGuard (protege as rotas internas)  
  pages/  
    login/       -> tela de login  
    layout/      -> shell com navegação lateral + logout  
    home/        -> visão geral (indicadores agregados)  
    cursos/      -> lista + formulário CRUD  
    servicos/    -> lista + formulário CRUD  
    pedidos/     -> lista + recálculo de risco + reagendamento + CRUD
```

3.3 Comunicação com a API

HttpClient com interceptor funcional que injeta `Authorization: Bearer <token>` em toda chamada, e um guard de rota que redireciona para o login quando não há sessão ativa. Aponta para `http://localhost:8080`, a mesma URL do backend da Parte 2.

3.4 Data binding e diretivas usadas

- **Interpolação** (`{{ }}`): títulos, contadores, mensagens de erro.
- **Property binding** (`[ ]`): `[disabled]`, `[class]`.
- **Event binding** (`()`): `(click)`, `(ngSubmit)`.
- **Two-way binding** (`[( )]`): `[(ngModel)]` nos formulários de login, curso, serviço e pedido.
- **Diretivas estruturais**: `*ngIf`, `*ngFor`.
- **Rotas**: `/login`, `/home`, `/cursos`, `/servicos`, `/pedidos`, protegidas por `authGuard`.

3.5 Feedback visual

Estados de carregamento, mensagens de erro amigáveis, badges coloridos de status de entrega e de nível de risco, confirmação antes de excluir um registro.

Validado rodando de verdade: `npm install` + `ng build` executados com sucesso, sem erros nem avisos.

4. Limitações conhecidas e próximos passos

- O backend Java cobre **hoje o contrato inteiro** do app (33 endpoints: núcleo AI Logistics + comunidade/gamificação/cuidador/indicação/marketplace de tutores/recomendações/tendências/LGPD) — não ficou restrito só ao núcleo. O que ainda é exclusivo do backend Python do time é a implementação em si, não a cobertura de funcionalidades.
- Dois bugs reais foram encontrados e corrigidos durante a validação ao vivo: uma `LazyInitializationException` ao serializar `topicosModulos` (corrigida com `fetch EAGER`), e um corpo JSON malformatado que devolvia 500 em vez de 400 (corrigido no `GlobalExceptionHandler`).
- Suíte de testes automatizados em JUnit para o backend Java e testes Karma/Jasmine para o dashboard Angular ficam como hardening final — mesmo critério já usado pelo grupo no restante do projeto.

Tudo neste documento foi validado rodando de verdade, não apenas revisado por leitura: backend Java compilado e executado (núcleo e comunidade), dashboard Angular compilado, e app Flutter com `dart analyze` limpo e as 85 verificações de `flutter test` passando.

5. Como executar o projeto completo

```
# 1. Backend Spring Boot (requer JDK 17+ e Maven)
cd backend-java
mvn spring-boot:run
# API em http://localhost:8080 | Swagger em http://localhost:8080/swagger-ui.html

# 2. Dashboard Angular (requer Node 18+)
cd dashboard
npm install
npm start
# Dashboard em http://localhost:4200 (login: admin / admin123)

# 3. App Flutter
cd ..
flutter pub get
flutter run -d chrome
```